

Wall Damage Detection Backend - Project Documentation

1. Project Overview (Non-Technical)

•**Goal**: To automatically detect, classify, and provide repair advice for wall damages using Artificial Intelligence.

This backend system acts as the "brain" of the application. When a user uploads a photo of a damaged wall, this system:

1. **Receives the image** from the mobile or web app.
2. **Analyzes the image** using advanced AI models to find cracks, paint flaking, or other issues.
3. **Calculates the severity** and estimates the repair cost.
4. **Consults an AI Expert (Gemini)** to generate step-by-step repair instructions.
5. **Returns a complete report** to the user in seconds.

2. Technical Architecture (For Developers)

The solution uses a hybrid **Node.js + Python** architecture to leverage the best of both worlds: robust web handling (Node/NestJS) and powerful AI libraries (Python).

Core Components

- **Web Server**: Built with **NestJS** (TypeScript). Handles API requests, file uploads (`Multer`), and client communication.
- **AI Engine**: Built with **Python**. Runs independent of the main server process to ensure stability.
- **The Bridge**: The NestJS `AppService` spawns a Python subprocess using `python-shell` to execute the analysis on demand.

Directory Structure

- `src/`: NestJS source code (Controllers, Services).
- `scripts/`: Python scripts for AI logic (`damage_analyzer.py`) and LLM integration (`LLM_explanations.py`).
- `models/`: Pre-trained AI models (`.pt` for YOLO, `.h5` for ResNet).
- `uploads/`: Temporary storage for user-uploaded images before analysis.
- `damage_analyzer.log`: Detailed runtime logs of the AI's decision-making process.

3. Deep Dive: AI & Research Models (For Researchers)

This project utilizes an **Ensemble Learning Strategy**, combining two distinct neural network architectures to maximize accuracy and robustness.

A. The Models

1. **YOLOv8 (You Only Look Once)**

- **File**: `models/best.pt``
- **Role**: Object Detection & Localization.
- **Strength**: Excellent at finding *where* the damage is and handling complex scenes with multiple damage types. It provides a precise bounding box and high-confidence classification.
- **Task**: It scans the image to draw boxes around specific defects (e.g., "crack_damages", "flaking_paint").

2. **ResNet-50 (Residual Network)**

- **File**: `models/resnet50_classifier_final.h5``
- **Role**: Whole-Image Classification.
- **Strength**: Deep understanding of textures and global image context. It looks at the *entire* image to categorize the dominant damage type.
- **Task**: It outputs a probability distribution across 4 classes: `['crack_damages', 'flaking_paint', 'missing_piece', 'water_damage']``.

B. Hybrid Decision Logic (The "Voting" System)

The python script (`scripts/damage_analyzer.py``) implements a smart decision algorithm to choose the final result. It doesn't just blindly trust one model.

• **Algorithm**:

1. **Run ResNet**: Get a baseline classification (e.g., "It looks like flaking paint" with 55% confidence).
2. **Run YOLO**: Check if precise objects are detected (e.g., "I found 2 cracks" with 85% confidence).
3. **The Decision Tree**:
 - **PRO YOLO**: If YOLO detects a known class with **high confidence (>50%)**, it overrides ResNet. Localized features are usually more accurate for specific defects.
 - **PRO YOLO (Tie-Breaker)**: If YOLO detects something valid, even if confidence is lower, but ResNet *also* has low confidence (<50%), we prefer YOLO because it found a specific feature.
 - **PRO ResNet**: If YOLO finds nothing (or low confidence) and ResNet is confident, we fall back to the global classification from ResNet.

C. Generative AI (LLM) Integration

Once the damage type is finalized, the system calls **Google Gemini 1.5 Flash** (`scripts/LLM_explanations.py``).

- **Prompt**: "Explain why {damage_type} occurs... list remediation steps... and materials."
- **Output**: Structured JSON containing ``causes`, `resolution`, `steps`, and `materials``.

- ****Fallback****: If the API call fails (network issue/quota), hardcoded static explanations are used as a failsafe.

4. Log Analysis Guide (`damage\_analyzer.log`)

The `damage\_analyzer.log` file is crucial for debugging and research optimization. Here is how to read a typical entry:

2025-07-14 21:02:44 - INFO - ResNet-50 probabilities: [[0.24, 0.25, 0.25, 0.25]]

2025-07-14 21:02:44 - INFO - YOLOv8 damage_type: flaking_paint, confidence: 0.417

2025-07-14 21:02:44 - INFO - ResNet-50 damage_type: flaking_paint, confidence: 0.252

2025-07-14 21:02:44 - INFO - Selected damage_type: flaking_paint

1. ****ResNet Probabilities****: Shows the raw softmax output. If these are all near 0.25, the model is "confused" or unsure.
2. ****YOLOv8 Output****: Shows what the object detector found. `None` means no bounding boxes were drawn.
3. ****Selected damage_type****: The FINAL decision made by the algorithm. ****This is what the user sees.****

•Common Errors to Watch:•

- `ERROR - Gemini API call failed: Subprocess failed...`: This often means the path to the python environment for the LLM script is incorrect or the API Key is missing.

5. Setup & Usage

Prerequisites

- Node.js (v18+)
- Anaconda (for managing Python environments)
- Google Gemini API Key

Configuration

1. ****Python Environment****: Ensure you have a conda environment named `wall\_damage` (for analysis) and `LLM\_env` (for Gemini).
2. ****API Keys****: Set `GEMINI\_KEY` in your system environment variables.
3. ****Path Configuration****:
 - Open `src/app.service.ts`.
 - Update `pythonPath` to point to your local `wall\_damage` environment python executable.
 - Open `scripts/damage\_analyzer.py`.
 - Update `conda\_path` to point to your local conda executable.

